Networked Context-Conditional Event Inference and Coordination for Multi-Feed Surveillance

HERMES Extension Working Draft

March 2, 2026

Abstract

We present a practical extension of HERMES for real-time multi-feed surveillance by coupling context-aware event inference with network-level message-passing optimization. The system combines dynamic event candidate routing, pluggable observation classifiers, context-conditional inhomogeneous Markov filtering with sliding-window updates, and policy-driven network transport simulation for inter-node coordination. On synthetic surveillance mesh scenarios with staged stress contexts (crowding, uplink loss, node degradation), the framework supports online adaptation and policy comparison between receding-horizon min-cost linear programming and decentralized backpressure routing. The implementation includes reproducible scripts, diagnostics, and weighted policy ranking for deployment-oriented decision making.

1 Introduction

Long-form video understanding in distributed camera systems depends on both perception quality and communication quality. In practice, coordination failures such as stale handoffs, duplicate alerts, or network congestion can dominate operational outcomes even when local model confidence is high.

We address this by integrating contextual event inference and network coordination evaluation in a single runnable pipeline. The goal is engineering utility: reproducible experiments, explicit KPIs, and policy selection under operator-defined priorities.

Contributions.

1. Dynamic event inference stack with taxonomy routing, configurable observation classifier fusion, and context-conditional Markov filtering.
2. Online runtime diagnostics with live transition/posterior visualization and interactive context switching.
3. Network message-passing simulator with two policies: receding-horizon min-cost LP and backpressure.
4. Surveillance-specific coordination KPIs (handoff success, stale ratio, duplicate alerts, recovery behavior).
5. Direction-aware weighted policy ranking for deployment-specific optimization targets.

2 System Overview

At each time window t, the pipeline performs:

1. candidate event selection via taxonomy,
2. model ranking and confidence scoring,
3. observation classifier fusion,
4. context-conditional Markov posterior update,
5. network transport simulation for inter-node coordination.

This yields a perception-to-transport loop suitable for end-to-end surveillance system evaluation.

3 Methods

3.1 Dynamic event inference

Given context/question input x_t, ranked event candidates produce confidence scores. Observation classifiers fuse model outputs with custom heuristics and prototypes to form likelihood-like scores $\tilde{p}_t(e)$.

3.2 Context-conditional Markov filtering

Posterior updates are:

$$\hat{p}_t = p_{t-1} T_{c_t,t}, \qquad p_t(e) \propto \hat{p}_t(e)\, \tilde{p}_t(e), \tag{1}$$

where $T_{c_t,t}$ may depend on ecological context, schedule, or provider logic.

The implementation supports:

- `window_size`: sliding-window refiltering,
- `markov_order`: higher-memory prior blending,
- optional symbolic transfer entropy diagnostics.

3.3 Network message passing model

We model a directed graph $G = (V, E)$ with time-varying edge capacities $C_e(t)$, delays d_e, and losses $p_e(t)$. Traffic commodities include source/destination/rate and surveillance metadata (`kind`, `ttl_steps`, `copies`).

Policy A: Receding-horizon min-cost LP.

$$\min_{x_{e,k}(t) \geq 0} \sum_{e,k} c_{e,k}(t)\, x_{e,k}(t) \tag{2}$$

subject to edge-capacity and queue-availability constraints.

Policy B: Backpressure.

$$P_{e,k}(t) = (Q_{u,k}(t) - Q_{v,k}(t)) + \beta \Delta \mathrm{dist}_{u \to v,k} - \gamma d_e. \tag{3}$$

Commodities with positive pressure are prioritized under edge capacities.

3.4 Weighted policy ranking

For each metric m, direction-aware normalization maps values to $[0, 1]$. Policy score:

$$S(\pi) = \frac{\sum_m w_m \cdot \mathrm{norm}_m(\pi)}{\sum_m w_m}, \tag{4}$$

where larger $S(\pi)$ indicates better fit to operator priorities.

4 Implementation

4.1 Core modules

- Event stack: `lavis/common/event_taxonomy.py`, `lavis/common/event_observation.py`, `lavis/common/eve` `stream_online.py`
- Network stack: `tools/network_message_passing_sim.py`, `tools/network_message_passing_kpi_report.p`

4.2 Run scripts

- Single surveillance run: `run_scripts/network_message_passing/surveillance_test.sh`
- Monte Carlo: `run_scripts/network_message_passing/surveillance_monte_carlo.sh`
- KPI and ranking report: `run_scripts/network_message_passing/kpi_report.sh`

5 Experimental Protocol

The included surveillance mesh scenario uses:

- 4 camera nodes, 2 edge nodes, 1 fusion node,
- traffic types: alert, track, handoff, heartbeat,
- context schedule: `normal` → `crowded` → `uplink_loss` → `cam2_failure`.

  Evaluation is performed via multi-seed Monte Carlo runs and aggregated KPI summaries.

6 Metrics

We report both packet-level and unique-message-level indicators:

- delivery ratio, drop ratio, average/p95 latency,
- unique delivery ratio,
- stale unique delivery ratio (TTL-based),

- duplicate delivery ratio,
- handoff success and timely handoff success,
- alert duplicate rate,
- backlog recovery after context transitions.

7 Representative Findings (Synthetic)

In representative synthetic surveillance runs with provided defaults, min-cost LP shows higher delivery ratio and lower p95 latency than backpressure, with improved timely handoff success and lower stale message rates. Backpressure remains a robust decentralized baseline and may be preferable under different objective weightings or centralized-solver constraints.

8 Limitations

1. Packet-level simulator is synthetic and does not yet replay real telemetry traces.
2. Min-cost LP is one-step receding-horizon optimization, not full finite-horizon global control.
3. The current framework does not jointly optimize model compute scheduling and network routing.

9 Conclusion

We introduced a runnable extension to HERMES that unifies context-aware event inference and surveillance coordination evaluation. The resulting framework supports online diagnostics, stress testing, reproducible policy comparison, and weighted policy selection for deployment-oriented system design.

Reproducibility Commands

```
bash run_scripts/network_message_passing/surveillance_monte_carlo.sh 20 320
bash run_scripts/network_message_passing/kpi_report.sh \
  "logs/network_message_passing/monte_carlo/run_*.json" \
  "logs/network_message_passing/monte_carlo_summary.json" \
  "handoff_timely_success_rate=4,p95_latency=3,stale_unique_ratio=3,delivery_ratio=2"
```